Kotlin

基础 28

OOP 29

集合 42

函数式编程

多线程 3

设计模式 9

协程 ···

流

Sequence

先看一个集合的问题

val list = mutableListOf<Int>().apply {
repeat(10) {
add(it)
}
}
var times = 0

list.filter {
println("filter " + times++)
it % 2 == 0
}.map {
println("map " + times++ + " number is " +
it)
it * 10
}.take(2).count()

打印结果：
filter 0
filter 1
filter 2
filter 3
filter 4
filter 5
filter 6
filter 7
filter 8
filter 9
map 10 number is 0
map 11 number is 2
map 12 number is 4
map 13 number is 6
map 14 number is 8

针对list中的10个元素，先遍历所有元素并执行

filter，然后执行map，最后执行take。
但是实际上，只需要符合条件的前两个元素

将集合转换为Sequence

val list = mutableListOf<Int>().apply {
repeat(10) {
add(it)
}
}
val sequence:Sequence<Int> = list.asSequence()
var times = 0

sequence.filter {
println("filter " + times++)
it % 2 == 0
}.map {
println("map " + times++ + " number is " + it)
it * 10
}.take(2).count()

打印结果：
filter 0
map 1 number is 0
filter 2
filter 3
map 4 number is 2

1、每个元素依次通过序列并执行完，执行完之后
才需要执行下一个

2、找到满足条件的第二个元素后，序列不再处
理数据

创建Sequence

1、通过asSequence将集合转化为Sequence

2、通过generateSequence创建

// 重载方法一
// T: startValue
// nextFunction : 如何生成下一个元素
public fun <T : Any> generateSequence(seed: T?, nextFunction: (T) -> T?): Sequence<T>

generateSequence(2) { it + 1 }.filter {
it % 2 == 0
}.map {
it * 10
}.take(2).count()

如果去掉take(2)，则该代码会无限循环下去

// 重载方法二
// nextFunction : 如何生成下一个，返回null则流结束
public fun <T : Any> generateSequence(nextFunction: () -> T?): Sequence<T>

generateSequence { Random.nextInt(1000) }.filter {
it % 2 == 0
}.map {
it * 10
}.take(2).count()

3、通过sequence创建

sequence {
println("start sequence")
yield(-1)
println("-1 yielded")
repeat(10) {
yield(it)
println("$it yielded")
}
}.filter {
println("filter $it")
it % 2 == 0
}.map {
println("map $it")
it * 10
}.take(2).count()

打印结果：
start sequence
filter -1
- 1 yielded
filter 0
map 0
0 yielded
filter 1
1 yielded
filter 2
map 2

1、yield是一个挂起函数，用于生成一个值并添
加到流。yield添加后挂起并执行【中间操作
符】，执行完后取消挂起，继续执行

2、sequence中仅能执行yield等少量挂起函数

末端操作符&中间操作符

Q: 如下代码打印什么？
generateSequence(2) { it + 1 }.filter {
println("$it filtered")
it % 2 == 0
}.take(2)
A: 什么也不打印

数据流只在需要使用的时候才会执行，判断是否
要使用的标准是【是否包含末端操作符】

1、当没有末端操作符时，中间操作符不工作，可
以理解为末尾有个塞子堵住了

2、当末尾有个末端操作符时，意味着这个流的
数据要被使用

怎么判断是末端还是中间操作符？

返回类型是Sequence的是中间操作符，否则是末
端操作符

有些末端操作符，会在数据收集完成后执行，例

如count()，而有些则是每新增一个执行一次，例
如forEach/collect

Flow

背景

1、流式编程的诉求

2、Sequence是同步的、阻塞式的，很多时候需
要放在协程中执行，但是Sequence仅支持yield
系列挂起函数，不支持其他挂起函数

Flow支持协程，支持流式操作的过程中切换线
程，临时挂起等操作，更灵活

创建

一、 通过flow创建

scope.launch {
flow {
repeat(10) {
method();
emit(it)
}
}.filter {
it % 2 == 0
}.map {
it * 10
}.take(2).collect {
println(it)
}
}

flow用法与Sequence类似，区别：
1、使用flow创建

2、可以调用suspend函数 method()
3、emit替代yield
4、collect替代forEach

二、通过flowOf创建 val coldFlow1 = flowOf(1, 2, 3)

三、通过asFlow创建 val flow1 = (1 .. 3).asFlow()

Channel

基本用法

// 创建通道
val channel = Channel<String>();
scope.launch {
// 将数据发送到通道
channel.send("test")
}

冷流无法从外部插入值，
需要借助Channel将数据导入流

方法一：Flow监听Channel

var channel = Channel<String>()

scope.launch {
// 将数据发送到通道
channel.send("test")
}

scope.launch {
flow {
// channel新增数据后，添加到flow
channel.consumeEach {
emit(it)
}
}.onEach {
// flow新增数据后，修改text
button.text = it
}.collect()
}

左例也是响应式编程的一个实例

channel.send是修改数据

修改数据时，自动触发flow的流，修改button显
示效果

方法二：将Channel转为Flow

var channel = Channel<String>()

scope.launch {
channel.consumeAsFlow().onEach {}.collect()
}

与共享流(热流)的不同

Channel.consumeAsFlow和receiveAsFlow可以将

Channel转化为一种特殊的热流，但是与热流是
有区别的

首先要明确一个点，consumeAsFlow和
receiveAsFlow是单例的，多次调用返回的是同一
个实例

consumeAsFlow不支持多次collect

val channel = Channel<Int>()
scope.launch {
var times = 0
while (true) {
channel.send(times++)
delay(1000)
}
}

val consumeAsFlow =
channel.consumeAsFlow()
scope.launch {
consumeAsFlow.collect {
println("collect 1 $it")
}
}

scope.launch {
consumeAsFlow.collect {
println("collect 2 $it")
}
}

第二次尝试collect失败，抛异常

receiveAsFlow支持多次collect，但是每次emit的
数据会轮询主歌发给collector

val channel = Channel<Int>()
scope.launch {
var times = 0
while (true) {
channel.send(times++)
delay(1000)
}
}

val receiveAsFlow = channel.receiveAsFlow()
scope.launch {
receiveAsFlow.collect {
println("collect 1 $it")
}
}

scope.launch {
receiveAsFlow.collect {
println("collect 2 $it")
}
}

scope.launch {
receiveAsFlow.collect {
println("collect 3 $it")
}
}

// 打印结果
collect 2 0
collect 3 1
collect 1 2
collect 2 3
collect 3 4
collect 1 5
collect 2 6
collect 3 7
collect 1 8
collect 2 9
collect 3 10

...

每次发射，哪个collector拿到，则该数据从
channel中删除，其他collector便拿不到了

注意对于MutableSharedFlow创建的热流，所有
的订阅者是可以同时收到发射的数据的

从Channel接收数据

val channel = Channel<Int>()

scope.launch {
var time = 0
while(true) {
delay(1000)
channel.send(time++)
}
}

scope.launch {
// 接收一个数据
val num = channel.receive()
}

scope.launch {
// 接收所有数据
channel.consumeEach {
println("consume1 $it")
}
channel.consumeEach {
println("consume2 $it")
}
}

// 打印结果：不固定
consume1 0
consume1 2
consume1 3
consume1 4
consume1 5
consume1 6
consume1 7

...

与receiveAsFlow一样，多个接收器时，如果某个
接收到数据后，数据从Chanel删除，其他接收器
无法接收到该数据

冷/热数据流

冷数据流

val coldFlow = flow {
var time = 0
while (true) {
delay(1000)
emit(time++)
}
}

scope.launch {
coldFlow.collect {
println(" collect a, num $it")
}
}

scope.launch {
delay(5000)
coldFlow.collect {
println(" collect b, num $it")
}
}

打印结果：
collect a, num 0
collect a, num 1
collect a, num 2
collect a, num 3
collect a, num 4
collect b, num 0
collect a, num 5
collect b, num 1
collect a, num 6
collect b, num 2
collect a, num 7
collect b, num 3
collect a, num 8

flow创建的是冷数据流，有几个特点

1、如果没有任何订阅（collect），冷流不发射

（emit）/处理任何数据

2、如果不再需要数据（例如take(2)结束），冷流
不再发射（emit）新数据

3、如果流停止发射（emit）数据，则收集器
（collect）也会结束
4、每次订阅（collect）会启动一个新的流（左
例可证）
5、冷流不能从外部插入（emit）值

热数据流

val hotFlow = MutableSharedFlow<Int>()
scope.launch{
var time = 0
while (true) {
delay(1000)
hotFlow.emit(time++)
}
}

scope.launch {
hotFlow.collect {
println(" collect a, num $it")
}
}

scope.launch {
delay(5000)
hotFlow.collect {
println(" collect b, num $it")
}
}

打印结果：
collect a, num 0
collect a, num 1
collect a, num 2
collect a, num 3
collect a, num 4
collect b, num 4
collect a, num 5
collect b, num 5
collect b, num 6
collect a, num 6
collect b, num 7
collect a, num 7

MutableSharedFlow创建热流，有几个特点

1、所有发射在收集器之间共享

2、热流永远不会停止，collect后面的代码不会执行
3、即使没有订阅（collect），热流也会被发射（emit）
4、如果订阅者（collect）不再需要数据，热流也会继续
发射（emit）
5、热流可以从外部插入（emit）值

冷转热

方法一：热流中订阅一个冷流

val coldFlow = flow {
while(true) {
emit(Random.nextInt(100))
}
}
val timer = MutableSharedFlow<Int>().apply {
scope.launch {
coldFlow.collect {
println(it)
}
}
}.asSharedFlow() // 转成不可变热流

方法二：使用shareIn

val coldFlow = flow {
while(true) {
emit(Random.nextInt(100))
}
}
val timer = coldFlow.shareIn(
scope = scope,

started = SharingStarted.Eagerly,
)

scope：指定冷流在哪个协程上下文中启动

started

Eagerly代表立即启动

Lazily代表首次被订阅后启动

WhileSubscribed代表有订阅者启动，无订阅者
停止

常用方法

OnStart/OnComplete/OnEach

// 示例代码
val coldFlow = flow {
repeat(5) {
emit(it)
println("$it emitted.")
}
}.onStart {
// 订阅前执行
emit(100)
println("100 emitted.")
}.onEach {
// 新数据发射执行
println("$it handled")
}.onCompletion {
// 订阅完成执行
emit(200)
println("200 emitted.")
}

scope.launch {
coldFlow.collect { }
}

打印结果：
100 handled
100 emitted.
0 handled
0 emitted.
1 handled
1 emitted.
2 handled
2 emitted.
3 handled
3 emitted.
4 handled
4 emitted.
200 emitted.

debounce()

只保留在指定时间段内最后一个数据项

适用场景：搜索的时候快速输入，每次输入变化
都发射，如果500ms没有输入则触发搜索

flow {
repeat(10) {
delay(100)
emit(it)
}
}.debounce(200).collect {
println(it)
}

所有数据发送间隔都是100，都不满足200ms条
件，此时输出最后一个数据

打印结果：9

launchIn()

public fun <T> Flow<T>.launchIn(scope: CoroutineScope): Job = scope.launch { collect() // tail-call
}

flow {
repeat(10) {
emit(it)
}
}.launchIn(scope)

等效于：
scope.launch {
flow {
repeat(10) {
emit(it)
}
}.collect()
}

tryEmit()

tryEmit支持在非协程中向Flow发射数据，核心特性
是非挂起，即不会阻塞当前协程

tryEmit尝试去Flow的缓存中写入，写入成功返回
true，如果没有缓存，则返回false

// 举例1
val sharedFlow = MutableSharedFlow<Int>
(replay = 1)

scope.launch {
sharedFlow.collect {
delay(1000)
println("collect $it")
}
}

Thread.sleep(1000)
println(" first " + sharedFlow.tryEmit(1))
println(" second " + sharedFlow.tryEmit(2))
println(" third " + sharedFlow.tryEmit(3))

// 打印结果
first true
second true
third false
collect 1
collect 2

主线程写入1、消费者处理1

主线程写入2、消费者还没处理完1，放入缓存
主线程写入3、消费者缓存已满，返回false

// 举例2，删除主线程睡眠
val sharedFlow = MutableSharedFlow<Int>
(replay = 1)

scope.launch {
sharedFlow.collect {
delay(1000)
println("collect $it")
}
}

// Thread.sleep(1000)
println(" first " + sharedFlow.tryEmit(1))
println(" second " + sharedFlow.tryEmit(2))
println(" third " + sharedFlow.tryEmit(3))

// 打印结果
first true
second true
third true
collect 3

主线程写入1、消费者还没启动，热流不处理，将1放入缓存供回放
主线程写入2、消费者还没启动，热流不处理，将2放入缓存供回放
主线程写入3、消费者还没启动，热流不处理，将3放入缓存供回放
消费者启动、主线程将回放缓存3给消费者

// 举例3，删除主线程睡眠，缓存改成
extraBufferCapacity
val sharedFlow = MutableSharedFlow<Int>
(extraBufferCapacity = 1)

scope.launch {
sharedFlow.collect {
delay(1000)
println("collect $it")
}
}

// Thread.sleep(1000)
println(" first " + sharedFlow.tryEmit(1))
println(" second " + sharedFlow.tryEmit(2))
println(" third " + sharedFlow.tryEmit(3))

// 打印结果
first true
second true
third true

主线程写入1、消费者还没启动，热流不处理，replay为0，直接丢弃
主线程写入2、消费者还没启动，热流不处理，replay为0，直接丢弃
主线程写入3、消费者还没启动，热流不处理，replay为0，直接丢弃
消费者启动、无任何数据

// 举例4，onBufferOverflow修改为
DROP_OLDEST
val sharedFlow = MutableSharedFlow<Int>
(replay = 1, onBufferOverflow =
BufferOverflow.DROP_OLDEST)

scope.launch {
sharedFlow.collect {
delay(1000)
println("collect $it")
}
}

Thread.sleep(1000)
println(" first " + sharedFlow.tryEmit(1))
println(" second " + sharedFlow.tryEmit(2))
println(" third " + sharedFlow.tryEmit(3))

// 打印结果
first true
second true
third true
collect 1
collect 3

主线程写入1，消费者处理1

主线程写入2，消费者还没处理完1，放入replay缓存
主线程写入3，消费者还没处理完1，替换replay缓存
消费者处理完1之后，处理3

使用场景

tryEmit可能丢失数据，一般不要使用默认的SUSPEND策略，而
配合DROP_OLDEST优先丢弃老数据

适用于大数据量，同时允许一定数据丢失的场景

缓冲区

冷流缓冲区：buffer()

背景：生产者效率大于消费者时，消费完成前生
产者挂起

// 生产者
val coldFlow = flow {
repeat(5) {
delay(100)
emit(it)
println("$it produced")
}
}

// 消费者
scope.launch {
coldFlow.collect {
delay(500)
println("$it consumed")
}
}

// 打印结果
0 produced
0 consumed
1 produced
1 consumed
2 produced
2 consumed
3 produced
3 consumed
4 produced
4 consumed

修改缓冲区大小

使用buffer，生产者不挂起，缓存起来
默认缓存64个

// 生产者
val coldFlow = flow {
repeat(5) {
delay(100)
println("$it produced")
emit(it)
}
}.buffer()

// 消费者
scope.launch {
coldFlow.collect {
delay(500)
println("$it consumed")
}
}

// 打印结果
0 produced
1 produced
2 produced
3 produced
4 produced
0 consumed
1 consumed
2 consumed
3 consumed
4 consumed

通过capacity指定缓冲区大小 buffer(capacity = 100)

热流缓冲区：创建时指定

热流缓冲区有点复杂，通过replay（默认0）和extraBufferCapacity（默认0）共同指定缓冲区大小，通过onBufferOverflow指定缓冲区满之后的策略举例：
val hotFlow = MutableSharedFlow<Int>(replay = 1, extraBufferCapacity = 0, onBufferOverflow = BufferOverflow.DROP_OLDEST)

热流的缓冲区大小 = replay + extraBufferCapacity = 默
认0，无缓存

val hotFlow = MutableSharedFlow<Int>()

scope.launch {
hotFlow.collect {
delay(1000)
println("consumer A : " + it)
}
}

scope.launch {
var time = 0;
while (true) {
delay(2000)
println("producer : " + ++time)
hotFlow.emit(time)
}
}

// 打印结果
producer : 1
consumer A : 1
producer : 2
consumer A : 2
producer : 3
consumer A : 3
producer : 4
consumer A : 4
producer : 5
consumer A : 5

热流默认缓冲区大小是1（冷流是0）

replay用于缓存需要回放的部分

背景：收集器收集晚时，无法获取数据

val hotFlow = MutableSharedFlow<Int>()

scope.launch {
launch {
hotFlow.collect {
println("cosumer 1 collect $it")
}
}
launch {
(1..4).forEach {
hotFlow.emit(it);
}
}
delay(1000)
launch {
hotFlow.collect {
println("consumer 2 collect $it")
}
}
}

// 打印结果
cosumer 1 collect 1
cosumer 1 collect 2
cosumer 1 collect 3
cosumer 1 collect 4

根据热流的特性，监听晚了的收集器无法接收到
数据

配置replay，可以把replay个条目缓存起来，保
障监听器可以至少获取到replay数量的数据

val hotFlow = MutableSharedFlow<Int>
(replay = 2)

scope.launch {
launch {
hotFlow.collect {
println("cosumer 1 collect $it")
}
}
launch {
(1..4).forEach {
hotFlow.emit(it);
}
}
delay(1000)
launch {
hotFlow.collect {
println("consumer 2 collect $it")
}
}
}

// 打印结果
cosumer 1 collect 1
cosumer 1 collect 2
cosumer 1 collect 3
cosumer 1 collect 4
consumer 2 collect 3
consumer 2 collect 4

collecter2可以拿到两个数据

extraBufferCapacity进行replay外的额外缓存

extraBufferCapacity常用于生产者速度大于消费
者速度的情况，跟冷流一样。一点小区别是缓存

大小 = replay + extraBufferCapacity

缓冲区满的处理策略

onBufferOverflow参数，指定缓冲区满时的处理
策略

// 冷流
val coldFlow = flow {}.buffer(onBufferOverflow = BufferOverflow.SUSPEND)
// 热流
val hotFlow = MutableSharedFlow<Int>(onBufferOverflow = BufferOverflow.SUSPEND)

BufferOverflow.SUSPEND 默认处理策略，buffer满时，生产者挂起等待

BufferOverflow.DROP_OLDEST 丢弃最旧的

BufferOverflow.DROP_LATEST 丢弃最新的

状态流

状态流MutableStateFlow是一个特殊的、常用的
Flow

// 初始值0
val stateFlow = MutableStateFlow<Int>(0)

// 与下面的热流等效
val hotFlow = MutableSharedFlow<Int>(replay = 1, onBufferOverflow = BufferOverflow.DROP_OLDEST)

常用场景

1、replay/缓存 = 1，只需要最近一次的数据
2、如果新数据来了，但是还没来得及处理，则
丢弃旧数据
上述场景式非常高频的数据刷新场景

value 由于状态流有初始值，且缓存为1，所以一定可以
获取到值，该值为value成员

// 创建时是针对value赋值
val stateFlow = MutableStateFlow(value = 0)

// value可以在协程外部赋值，实际是调用的tryEmit
scope.launch {
stateFlow.collect {
delay(2000)
println("consumer $it")
}
}

repeat(100) {
Thread.sleep(1000)
stateFlow.value = it + 100
}

冷转状态流 与冷转热的shareIn类似，冷转状态流通过stateIn val coldFlow = flowOf(1..4)

coldFlow.stateIn(scope, SharingStarted.Eagerly, initialValue = 0)

处理异常

不建议在flow内部通过try-catch块捕获

val coldFlow = flow {
try {
var time = 0;
while (true) {
delay(1000)
println("emit $time")
emit(time++)
if (time == 5) {
error("Exception occured")
}
}
} catch (e : Exception) {
println("Exception caught")
}
}

不符合异常透明原则，流的消费者无法感知流的
内部错误，并且针对内部错误执行诸如重试等操
作

emit是挂起函数，会把消费者的异常异常掉！！

// 举例，如果消费者的代码这么写，则消费者抛
出的异常默认被生产者捕获，导致无法发现
scope.launch {
coldFlow.collect {
error("consumer exception")
}
}

推荐写法：通过catch

val coldFlow = flow {
var time = 0;
while (true) {
delay(1000)
println("emit $time")
emit(time++)
if (time == 5) {
error("Exception occured")
}
}
}

scope.launch {
coldFlow.catch {
println("Exception caught")
}.collect {
println("collect $it")
}
}

// 打印结果
emit 0
collect 0
emit 1
collect 1
emit 2
collect 2
emit 3
collect 3
emit 4
collect 4
Exception caught

从打印结果看，catch()捕获异常之后，整个流也
就停止了，异常被捕获

catch捕获到异常，不会导致该协程作用域内的
子协程均停止执行

val coldFlow = flow {
var time = 0;
while (true) {
delay(1000)
println("emit $time")
emit(time++)
if (time == 5) {
error("Exception occured")
}
}
}

scope.launch {
coldFlow.catch {
println("Exception caught")
}.collect {
println("collect $it")
}
}

scope.launch {
var time = 100;
while (true) {
delay(1000)
println("other sub-coroutine ${time++}")
}
}

// 打印结果
other sub-coroutine 100
emit 0
collect 0
other sub-coroutine 101
emit 1
collect 1
other sub-coroutine 102
emit 2
collect 2
other sub-coroutine 103
emit 3
collect 3
other sub-coroutine 104
emit 4
collect 4
Exception caught
other sub-coroutine 105
other sub-coroutine 106
other sub-coroutine 107
other sub-coroutine 108

流停止后，其他子协程继续执行

通过retry重试

val coldFlow = flow {
var time = 0;
while (true) {
delay(1000)
println("emit $time")
emit(time++)
if (time == 5) {
error("Exception occured")
}
}
}

scope.launch {
coldFlow.retry().catch {
println("Exception caught")
}.collect {
}
}

// 打印结果
emit 0
emit 1
emit 2
emit 3
emit 4
emit 0
emit 1
emit 2
emit 3
emit 4
emit 0
emit 1
emit 2
emit 3
emit 4
emit 0
emit 1
emit 2
emit 3
emit 4
emit 0
emit 1

retry() 无限重试，注意只要重试不结束，catch
块里的日志不会打印

val coldFlow = flow {
var time = 0;
while (true) {
delay(1000)
println("emit $time")
emit(time++)
if (time == 5) {
error("Exception occured")
}
}
}

scope.launch {
coldFlow.retry(retries = 3).catch {
println("Exception caught")
}.collect {
}
}

// 打印结果
emit 0
emit 1
emit 2
emit 3
emit 4
emit 0
emit 1
emit 2
emit 3
emit 4
emit 0
emit 1
emit 2
emit 3
emit 4
emit 0
emit 1
emit 2
emit 3
emit 4
Exception caught

retry(3)指定最大重试次数，重试最后一次如果还
捕获到异常，执行catch块中的代码

补充：如果retry最大次数后，没有catch块，则
异常向上抛出

val coldFlow = flow {
var time = 0;
while (true) {
delay(1000)
println("emit $time")
emit(time++)
if (time == 5) {
error("Exception occured")
}
}
}

scope.launch {
var retryTime = 0
coldFlow.retry {
retryTime++
println("retry $retryTime times")
true
}.collect {
}
}

// 打印结果
emit 0
emit 1
emit 2
emit 3
emit 4
retry 1 times
emit 0
emit 1
emit 2
emit 3
emit 4
retry 2 times
emit 0
emit 1
emit 2
emit 3
emit 4
retry 3 times
.... 无限重试

retry最后一个参数，配置是否继续重试的
predicate
本例中永远返回true，且没有配置retries，则无
限重试

val coldFlow = flow {
var time = 0;
while (true) {
delay(1000)
println("emit $time")
emit(time++)
if (time == 5) {
error("Exception occured")
}
}
}

scope.launch {
var retryTime = 0
coldFlow.retry(retries = 3) {
retryTime++
println("retry $retryTime times")
retryTime != 1
}.collect {
}
}

// 打印结果
emit 0
emit 1
emit 2
emit 3
emit 4
retry 1 times
Exception Caught

retries配置最大重试次数是3

但是大括号内配置的predicate仅重试一次
最终重试次数 = min{retries, predicate time}

此外，由于内部没有catch，故异常抛出给协程

域，交由协程域的ExceptionHandler捕获

切换上下文

通过flowOn切换上下文，实践中较少使用

默认情况下一个流处理器各步骤运行在同一线程

scope.launch {
(1..3).asFlow().onStart {
println("onStart ${getCurrentThread()}")
}.onEach {
println("onEach ${getCurrentThread()}")
}.map {
println("map ${getCurrentThread()}")
it
}.collect {
println("collect $it
${getCurrentThread()}")
}
}

// 打印结果
onStart pool-1-thread-1
onEach pool-1-thread-1
map pool-1-thread-1
collect 1 pool-1-thread-1
onEach pool-1-thread-1
map pool-1-thread-1
collect 2 pool-1-thread-1
onEach pool-1-thread-1
map pool-1-thread-1
collect 3 pool-1-thread-1

flowOn仅影响上面的处理

scope.launch {
(1..3).asFlow().onStart {
println("onStart ${getCurrentThread()}")
}.onEach {
println("onEach ${getCurrentThread()}")
}.flowOn(dispatcher).map {
println("map ${getCurrentThread()}")
it

}.flowOn(Dispatchers.Default).collect {
println("collect $it
${getCurrentThread()}")
}
}

// 打印结果
onStart pool-1-thread-2
onEach pool-1-thread-2
onEach pool-1-thread-2
onEach pool-1-thread-2
map DefaultDispatcher-worker-1
map DefaultDispatcher-worker-1
map DefaultDispatcher-worker-1
collect 1 pool-1-thread-3
collect 2 pool-1-thread-3
collect 3 pool-1-thread-3

通过flowOn使map执行在default线程
由于线程切换，流由之前的逐个执行切换为逐函
数执行

类图

语法糖一 37

语法糖二 17

泛型 33

内联函数 19

数据类 13

委托 14